

■ 第六课： 中断与数码管动态显示

6.1 C语言的数组

6.2 if语句

6.3 switch语句

6.4 数码管的动态显示

6.5 单片机中断系统

6.1 数组

- 数组是一组变量，这组变量需要满足三个条件：

相同的数据类型；相同的名字；在存储器中是被连续存放的。

- 数组的声明和初始化

数组类型 数组名[数组长度]

数组类型 数组名[数组长度]={初值列表};

```
9 unsigned char code LedChar[]={
10     0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92, 0x82, 0xF8,
11     0x80, 0x90, 0x88, 0x83, 0xC6, 0xA1, 0x86, 0x8E
12 };
13
```

- 多维数组a[2][3]

2行3列的多维数组

6.1 数组

数组的使用和赋值注意事项

- 1、区分数组下标和数组元素个数的概念
引用下标索引从0-15，数组元素个数是16个
- 2、数组元素下标可以**常量或变量或表达式**，初始化必须常量
- 3、数组整体赋值只能在初始化进行，程序执行只能单个元素赋值，不能再对全体数组进行赋值。

```
8  
9 unsigned char code LedChar[]={  
10     0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92, 0x82, 0xF8,  
11     0x80, 0x90, 0x88, 0x83, 0xC6, 0xA1, 0x86, 0x8E  
12 };  
13
```

定义数组的时候，中括号必须是常量，引用元素的时候下表可以是变量，比如**LedChar[i]**，其中i取值可以从**0-15**。

6.2 if语句

- if语句的默认形式:

```
if(条件表达式)
{
    语句1;
}
```

- if...else语句

```
if(条件表达式)
{
    语句1;
}
else
{
    语句2;
}
```

```
29
30 while(1)
31 {
32     if(TF0==1)
33     {
34         TF0 = 0;
35         TH0 = 0xB8;
36         TL0 = 0x00;
37         cnt++;
38         if(cnt>=50)
39         {
40             cnt = 0;
41             P0 = LedChar[sec];
42             sec++;
43             if(sec>=16)
44             {
45                 sec = 0;
46             }
47         }
48     }
49 }
50
51 }
```

- **if...else if**语句
- if(表达式1) {语句1;}
- else if(表达式2) {语句2;}
- else if(表达式3) {语句3;}
-
- else {语句n;}

6.3 switch语句的用法

■ switch(表达式)

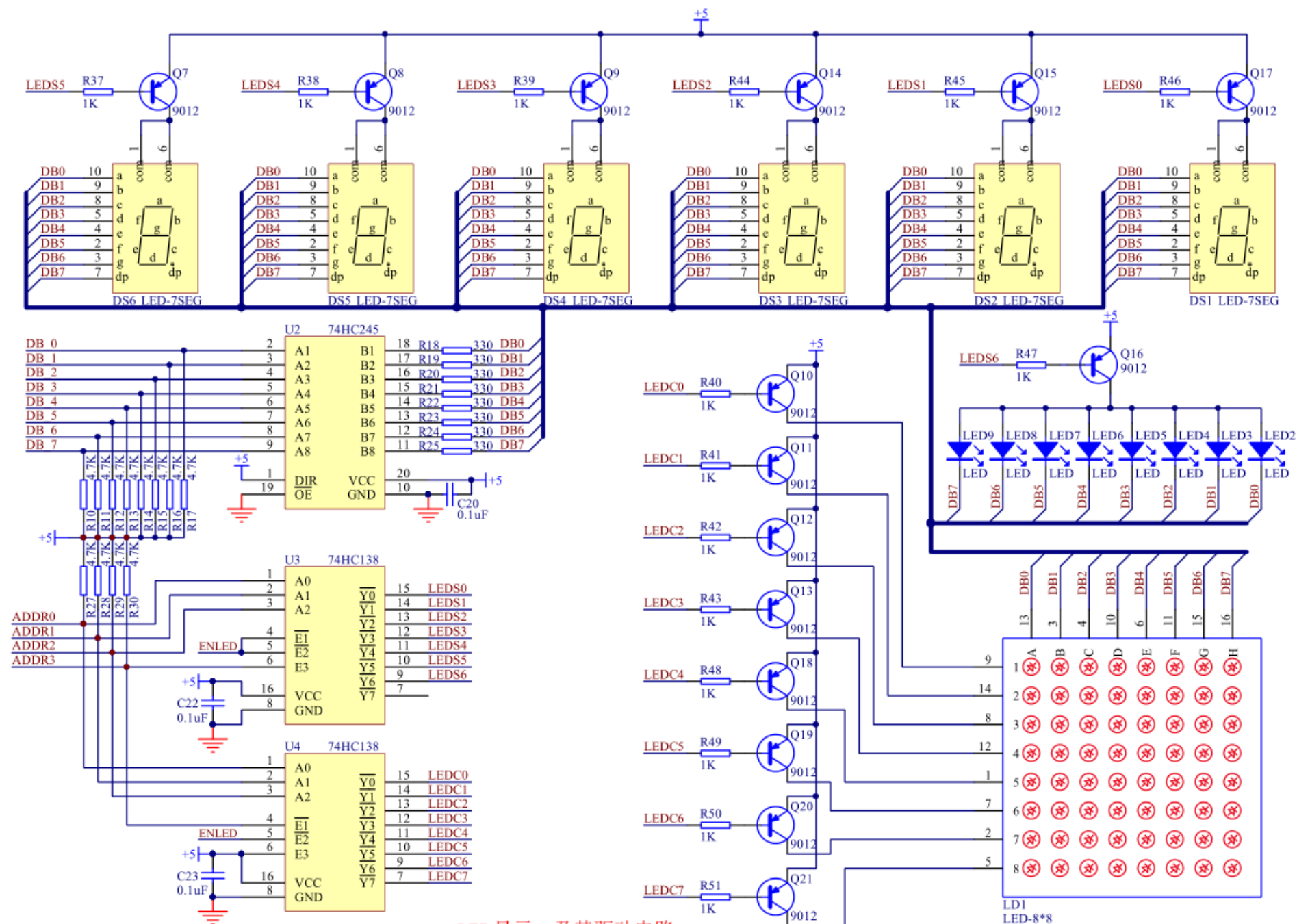
```
{  
    ● case 常量表达式1: 语句1;  
    ● case 常量表达式2: 语句2;  
    ● ... ...  
    ● case 常量表达式n: 语句n;  
    ● default : 语句n+1;  
}
```

■ switch(表达式)

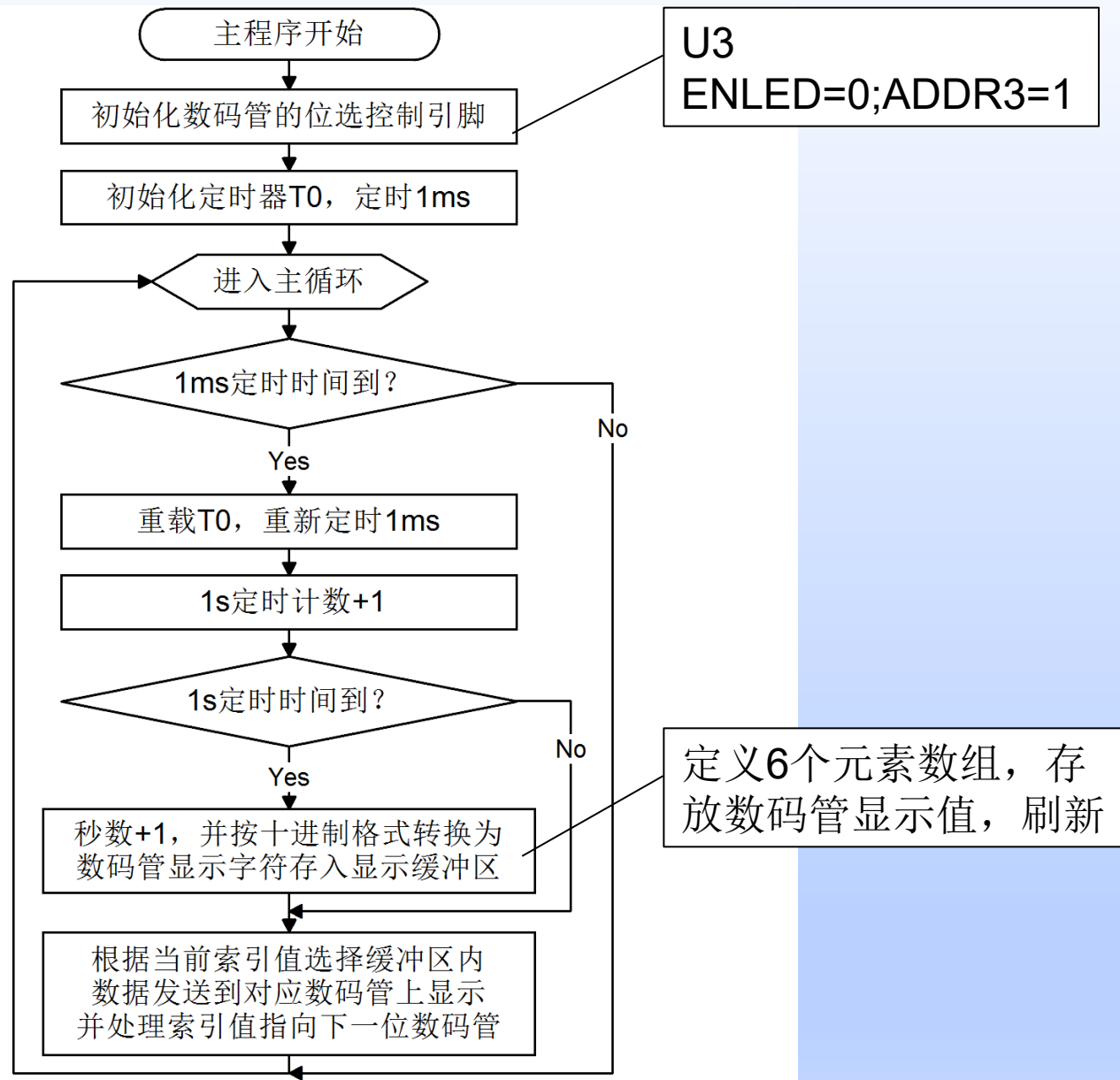
```
{  
    ● case 常量表达式1: 语句1; break;  
    ● case 常量表达式2: 语句2; break;  
    ● ... ...  
    ● case 常量表达式n: 语句n; break;  
    ● default : 语句n+1; break;  
}
```

6.4 动态显示的基本原理

- 动态显示:
- 多个数码管显示数字的时候, 我们实际上是轮流点亮数码管(一个时刻只有一个数码管是亮的), 利用人眼的视觉暂留现象(也叫余晖效应)。
- $1\text{ms} \times 6 = 6\text{ms}$
- $6\text{ms} < 10\text{ms}$



LED显示、及其驱动电路



数码管动态显示秒表流程图

```
1 #include<reg52.h>
2
3 sbit ADDR0 = P1^0;
4 sbit ADDR1 = P1^1;
5 sbit ADDR2 = P1^2;
6 sbit ADDR3 = P1^3;
7 sbit ENLED = P1^4;
8
9 unsigned char LedChar[16]={
10     0xC0,0xF9,0xA4,0xB0,0x99,0x92,0x82,0xF8,
11     0x80,0x90,0x88,0x83,0xC6,0xA1,0x86,0x8E
12 };
13 unsigned char LedBuff[6]={    //数组显示数码管值，经常变化，不能在FLASH中，应该在RAM中。
14     0xFF,0xFF,0xFF,0xFF,0xFF,0xFF
15 };
16
17 void main() {
18     unsigned int cnt = 0;//char最大255，1ms*1000=1s，最大要1000，因此定义int型
19     unsigned long sec = 0;
20     unsigned char i = 0;//定义第几个数码管
21
22     ENLED = 0;
23     ADDR3 = 1;
24
25     TMOD = 0x01;
26     TH0 = 0xFC;
27     TLO = 0x67;
28     TR0 = 1;
```

```

30 while(1)
31 {
32     if(TF0 ==1)
33     {
34         TF0 = 0;
35         TH0 = 0xFC;
36         TL0 = 0x67;
37         cnt++;
38         if(cnt >= 1000)
39         {
40             cnt = 0;
41             sec++;
42             LedBuff[0] = LedChar[sec%10]; //显示个位
43             LedBuff[1] = LedChar[sec/10%10]; //显示十位
44             LedBuff[2] = LedChar[sec/100%10];
45             LedBuff[3] = LedChar[sec/1000%10];
46             LedBuff[4] = LedChar[sec/10000%10];
47             LedBuff[5] = LedChar[sec/100000%10];
48         }
49         if(i == 0)
50             {ADDR2=0;ADDR1=0;ADDR0=0; i++; P0=LedBuff[0];}
51         else if(i == 1)
52             {ADDR2=0;ADDR1=0;ADDR0=1; i++; P0=LedBuff[1];}
53         else if(i == 2)
54             {ADDR2=0;ADDR1=1;ADDR0=0; i++; P0=LedBuff[2];}
55         else if(i == 3)
56             {ADDR2=0;ADDR1=1;ADDR0=1; i++; P0=LedBuff[3];}
57         else if(i == 4)
58             {ADDR2=1;ADDR1=0;ADDR0=0; i++; P0=LedBuff[4];}
59         else if(i == 5)
60             {ADDR2=1;ADDR1=0;ADDR0=1; i=0; P0=LedBuff[5];}
61     }
62 }
63
64 }
65

```

```
if(i == 0)
{ADDR2=0;ADDR1=0;ADDR0=0; i++; P0=LedBuff[0];}
else if(i == 1)
{ADDR2=0;ADDR1=0;ADDR0=1; i++; P0=LedBuff[1];}
else if(i == 2)
{ADDR2=0;ADDR1=1;ADDR0=0; i++; P0=LedBuff[2];}
else if(i == 3)
{ADDR2=0;ADDR1=1;ADDR0=1; i++; P0=LedBuff[3];}
else if(i == 4)
{ADDR2=1;ADDR1=0;ADDR0=0; i++; P0=LedBuff[4];}
else if(i == 5)
{ADDR2=1;ADDR1=0;ADDR0=1; i=0; P0=LedBuff[5];}
```

```
switch(i)
{
    case 0:ADDR2=0;ADDR1=0;ADDR0=0;i++;P0=LedBuff[0];break;
    case 1:ADDR2=0;ADDR1=0;ADDR0=1;i++;P0=LedBuff[1];break;
    case 2:ADDR2=0;ADDR1=1;ADDR0=0;i++;P0=LedBuff[2];break;
    case 3:ADDR2=0;ADDR1=1;ADDR0=1;i++;P0=LedBuff[3];break;
    case 4:ADDR2=1;ADDR1=0;ADDR0=0;i++;P0=LedBuff[4];break;
    case 5:ADDR2=1;ADDR1=0;ADDR0=1;i=0;P0=LedBuff[5];break;
    default:break;
}
```

数码管消隐

■ 产生背景:

数码管位选切换过渡阶段造成的段选的瞬态显示。

■ 解决方法:

- 1、每次刷新完，延时；
- 2、切换之前关闭位选或者段选
- 3、切换后一次性打开。

```
P0 = 0xff;
switch(i)
{
    case 0: ADDR2=0; ADDR1=0; ADDR0=0; i++; P0=LedBuff[0]; break;
    case 1: ADDR2=0; ADDR1=0; ADDR0=1; i++; P0=LedBuff[1]; break;
    case 2: ADDR2=0; ADDR1=1; ADDR0=0; i++; P0=LedBuff[2]; break;
    case 3: ADDR2=0; ADDR1=1; ADDR0=1; i++; P0=LedBuff[3]; break;
    case 4: ADDR2=1; ADDR1=0; ADDR0=0; i++; P0=LedBuff[4]; break;
    case 5: ADDR2=1; ADDR1=0; ADDR0=1; i=0; P0=LedBuff[5]; break;
    default: break;
}
```

```
ENLED=1;
switch(i)
{
    case 0: ADDR2=0; ADDR1=0; ADDR0=0; i++; P0=LedBuff[0]; break;
    case 1: ADDR2=0; ADDR1=0; ADDR0=1; i++; P0=LedBuff[1]; break;
    case 2: ADDR2=0; ADDR1=1; ADDR0=0; i++; P0=LedBuff[2]; break;
    case 3: ADDR2=0; ADDR1=1; ADDR0=1; i++; P0=LedBuff[3]; break;
    case 4: ADDR2=1; ADDR1=0; ADDR0=0; i++; P0=LedBuff[4]; break;
    case 5: ADDR2=1; ADDR1=0; ADDR0=1; i=0; P0=LedBuff[5]; break;
    default: break;
}
ENLED=0;
```

数码管问题2

■ 产生背景:

数码管每次过1s，其他显示位也会抖动一下。复杂的加减乘除，消耗单片机时间比较长。

```
if(cnt >= 1000)
{
    cnt = 0;
    sec++;
    LedBuff[0] = LedChar[sec%10]; //显示个位
    LedBuff[1] = LedChar[sec/10%10]; //显示十位
    LedBuff[2] = LedChar[sec/100%10];
    LedBuff[3] = LedChar[sec/1000%10];
    LedBuff[4] = LedChar[sec/10000%10];
    LedBuff[5] = LedChar[sec/100000%10];
}
```

```
P0 = 0xFF;
switch(i)
{
    case 0: ADDR2=0; ADDR1=0; ADDR0=0; i++; P0=LedBuff[0]; break;
    case 1: ADDR2=0; ADDR1=0; ADDR0=1; i++; P0=LedBuff[1]; break;
    case 2: ADDR2=0; ADDR1=1; ADDR0=0; i++; P0=LedBuff[2]; break;
    case 3: ADDR2=0; ADDR1=1; ADDR0=1; i++; P0=LedBuff[3]; break;
    case 4: ADDR2=1; ADDR1=0; ADDR0=0; i++; P0=LedBuff[4]; break;
    case 5: ADDR2=1; ADDR1=0; ADDR0=1; i=0; P0=LedBuff[5]; break;
    default: break;
}
```

6.5 中断系统

- 比如说我正在厨房用煤气烧一壶水，这样就只能守在厨房里，苦苦等着水开——如果水溢出来浇灭了煤气，有可能就要发生一场灾难了。
- 门外忽然又传来了铿锵有力的歌声，我最喜欢的天龙八部要开演了，听着水壶发出“咕嘟咕嘟”的声音，我清楚：除非等到水开，否则没有我享受人生的时候。
- 这个场景跟中断有什么关系呢？

- 在这个场景中，我是唯一具有处理能力的主体，不管是烧水、还是看电视，同一个时间点上我只能干一件事情。但是，在我专心致志干一件事情时，总有许多或紧迫或不紧迫的事情突然出现在面前，都需要去关注，有些还需要我停下手头的工作马上去处理。只有在处理完之后，方能回头完成先前的任务。
- 中断机制不仅赋予了我处理意外情况的能力，如果我能充分发挥这个机制的妙用，就可以“同时”完成多个任务了。

- 事实上烧水需要**10**分钟完成，但是提下水壶和关煤气我只要几秒钟就可以完成。为了这几秒，我需要在厨房等候**10**分钟。如果使用闹钟定时**10**分钟，**10**分钟一到，闹钟就会提醒我该去关煤气了，那么我就可以去安心看电视了。
- 实际上就是用了闹钟这样一个中断信号来提示我完成提水壶和关煤气的任务。

IE—中断使能寄存器的位分配（地址0xA8、可位寻址）

位	7	6	5	4	3	2	1	0
符号	EA	--	ET2	ES	ET1	EX1	ET0	EX0
复位值	0	--	0	0	0	0	0	0

IE—中断使能寄存器的位描述

位	符号	描述
7	EA	总中断使能位，相当于总开关
6	--	--
5	ET2	定时器2中断使能
4	ES	串口中断使能
3	ET1	定时器1中断使能
2	EX1	外部中断1使能
1	ET0	定时器0中断使能
0	EX0	外部中断0使能

```
1#include<reg52.h>
2
3sbit ADDR0 = P1^0;
4sbit ADDR1 = P1^1;
5sbit ADDR2 = P1^2;
6sbit ADDR3 = P1^3;
7sbit ENLED = P1^4;
8
9unsigned char LedChar[16]={
10    0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92, 0x82, 0xF8,
11    0x80, 0x90, 0x88, 0x83, 0xC6, 0xA1, 0x86, 0x8E
12};
13unsigned char LedBuff[6]={    //数组显示数码管值，经常变化，不能在FLASH中，应该在RAM中。
14    0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF
15};
16unsigned char flag1s = 0;
17unsigned int cnt = 0; //char最大255, 1ms*1000=1s, 最大要1000, 因此定义int型
18unsigned char i = 0; //定义第几个数码管
19
20void main() {
21
22    unsigned long sec = 0;
23    EA = 1;
24    ET0 = 1;
25    ENLED = 0;
26    ADDR3 = 1;
27
28    TMOD = 0x01;
29    TH0 = 0xFC;
30    TL0 = 0x67;
31    TR0 = 1;
```

```
33 while(1)
34 {
35     if(flag1s ==1) //主函数在判断是否到1s
36     {
37         flag1s =0;
38         sec++;
39         LedBuff[0] = LedChar[sec%10]; //显示个位
40         LedBuff[1] = LedChar[sec/10%10]; //显示十位
41         LedBuff[2] = LedChar[sec/100%10];
42         LedBuff[3] = LedChar[sec/1000%10];
43         LedBuff[4] = LedChar[sec/10000%10];
44         LedBuff[5] = LedChar[sec/100000%10];
45     }
46 }
47
48 }
```

```
49 void InterruptTime0() interrupt 1 //中断函数后面有关键字和数字；不需要调用，达到条件，自动进入
50 {
51     TH0 = 0xFC; //进入定时器中断硬件自动清零
52     TL0 = 0x67;
53     cnt++;
54     if(cnt >= 1000)
55     {
56         cnt = 0;
57         flag1s = 1;
58     }
59
60     P0 = 0xFF;
61     switch(i)
62     {
63         case 0: ADDR2=0; ADDR1=0; ADDR0=0; i++; P0=LedBuff[0]; break;
64         case 1: ADDR2=0; ADDR1=0; ADDR0=1; i++; P0=LedBuff[1]; break;
65         case 2: ADDR2=0; ADDR1=1; ADDR0=0; i++; P0=LedBuff[2]; break;
66         case 3: ADDR2=0; ADDR1=1; ADDR0=1; i++; P0=LedBuff[3]; break;
67         case 4: ADDR2=1; ADDR1=0; ADDR0=0; i++; P0=LedBuff[4]; break;
68         case 5: ADDR2=1; ADDR1=0; ADDR0=1; i=0; P0=LedBuff[5]; break;
69         default: break;
70     }
71 }
72
```

void InterruptTime0() interrupt 1

中断函数后面有关键字和数字

公式： $X*8+3=0x000B$ 计算 $X=1$

中断查询序列

中断函数 编号	中断名称	中断标志 位	中断使能 位	中断向量 地址	默认优先 级
0	外部中断	IE0	EX0	0x0003	1(最高)
1	T0	TF0	ET0	0x000B	2
2	外部中断	IE1	EX1	0x0013	3
3	T1	TF1	ET1	0x001B	4
4	UART	TI/RI	ES	0x0023	5
5	T2	TF2/EXF2	ET2	0x002B	6

中断优先级

- 1当设置为默认中断**固有优先级**时：

当几个中断同时发生时，则先处理中断**优先级高**的中断程序，在处理任意中断期间发生中断，都不会响应。

- 2当配置了中断优先级，即**抢占优先级**

同时发生中断，优先级高的先响应，在处理任意中断时，发生同级别或低级的中断，则不响应，发生优先级更高的中断时，则先处理高优先级中断，处理完毕，再回来处理当前中断。

1 中断固有优先级的说明

- 当设置为默认固有中断优先级时：
假设“水开”默认优先级高于“门铃”优先级，当先听到“水开”则先处理“水开”这件事，当先听到“门铃”则先处理“门铃”这件事。
只有当两件事同时发生时，则会先去处理“水开”，再处理“门铃”。

中断函数 编号	中断名称	中断标志 位	中断使能 位	中断向量 地址	默认优先 级
0	外部中断	IE0	EX0	0x0003	1(最高)
1	T0	TF0	ET0	0x000B	2
2	外部中断	IE1	EX1	0x0013	3
3	T1	TF1	ET1	0x001B	4
4	UART	TI/RI	ES	0x0023	5
5	T2	TF2/EXF2	ET2	0x002B	6

2抢占中断优先级的说明

- 当人为配置了抢占中断优先级
- 配置“水开”为高优先级，则当“水开”事件发生时，直接处理“水开”，在处理“水开”的过程中，即使“门铃”响了，也不会去理会。
- 当“门铃”响了，正在走向门口时，这个时候，“水开”事件又发生了，那么从门口转向，先去处理“水开”，处理好了后，再回头到门口，处理“门铃”事件。

IP—中断优先级寄存器的位分配（地址0xB8、可位寻址）

位	7	6	5	4	3	2	1	0
符号	--	--	PT2	PS	PT1	PX1	PT0	PX0
复位值	--	--	0	0	0	0	0	0

IP—中断优先级寄存器的位描述

位	符号	描述
7	--	保留
6	--	保留
5	PT2	定时器2中断优先级控制位
4	PS	串口中断优先级控制位
3	PT1	定时器1中断优先级控制位
2	PX1	外部中断1中断优先级控制位
1	PT0	定时器0中断优先级控制位
0	PX0	外部中断0中断优先级控制位

作业

■ 手写（信笺纸）【左上角写个人信息】：

1、什么是固有优先级？什么是抢占优先级？

2、“中断可以嵌套吗”？实现中断嵌套需要满足哪些条件？请试着解释为什么要限制嵌套层数？

■ 操作练习：

3、独立完成编程本章实验。